

Sequential Task:

Notification System

Object-Oriented Programming and GUI

March 15, 2026

1. Declare a public interface **Notification** that will define a common contract for different notification delivery mechanisms.
 - (a) Add the method **send(String message)**, whose purpose is to send the provided notification content.
 - Accept the parameter **message**, representing the content of the message to be sent.
 - Do not implement the method logic in the interface, but leave its implementation to the classes that implement it.
 - (b) Add the method **getType()**, whose purpose is to return the type of notification handled by a given implementation.
 - Set the return type to **String**.
 - Do not implement the method logic in the interface, but leave its implementation to the classes that implement it.
2. Declare an abstract class **BaseNotification**, which implements the interface **Notification** and provides shared notification-sending logic for subclasses.
 - (a) Add a private immutable field **recipient**, which stores the identifier of the notification recipient.
 - (b) Implement the constructor **BaseNotification(String recipient)**, whose purpose is to initialize the field storing the notification recipient.
 - i. Accept the parameter **recipient**, representing the notification recipient.
 - ii. Assign the provided value to the field **recipient**.
 - (c) Add a public accessor method **getRecipient()**, which returns the value of the field **recipient**.
 - (d) Implement the method **send(String message)**, which performs the general algorithm for sending a notification.
 - i. Call the method **formatMessage(message)** to prepare the formatted content of the message and store the result in the variable **formatted**.
 - ii. Call the method **doSend(formatted)** to pass the prepared message to the appropriate sending mechanism.

- (e) Declare a protected abstract method `formatMessage(String message)`, responsible for preparing the final content of the message before sending.
 - Accept the parameter `message`, representing the original notification content.
 - Return the formatted version of the message as a `String`.
 - Leave the implementation of this method to the inheriting classes.
 - (f) Declare a protected abstract method `doSend(String formatted)`, responsible for actually sending the prepared message.
 - Accept the parameter `formatted`, representing the already formatted notification content.
 - Leave the implementation of the sending mechanism to the inheriting classes.
3. Declare the class `EmailNotification`, which represents a notification sent in the form of an email message, and set it as a class inheriting from the abstract class `BaseNotification`.
- (a) Implement the constructor `EmailNotification(String email)`, whose purpose is to pass the recipient's email address to the base class.
 - i. Accept the parameter `email`, representing the recipient's email address.
 - ii. Call the constructor of the superclass `BaseNotification`, passing the parameter `email`.
 - (b) Override the method `getType()` to specify the type of notification handled by this class.
 - Return the string "EMAIL" as the notification type identifier.
 - (c) Override the method `formatMessage(String message)`, whose purpose is to prepare the email message content in an appropriate textual format.
 - Accept the parameter `message`, representing the content of the notification.
 - Retrieve the recipient's address by calling the method `getRecipient()`.
 - Build a string containing the subject section, the recipient, and the body of the message.
 - Return the created text as the formatted message.
 - (d) Override the method `doSend(String formatted)`, whose purpose is to simulate sending an email message.
 - Accept the parameter `formatted`, representing the previously prepared message content.
 - Display the message in the console using `System.out.println`.
 - Prefix the content with [EMAIL] to indicate the type of notification.
4. Declare the class `SmsNotification`, which represents a notification sent in the form of an SMS message, and set it as a class inheriting from the abstract class `BaseNotification`.
- (a) Implement the constructor `SmsNotification(String phoneNumber)`, whose purpose is to pass the recipient's phone number to the base class.

- i. Accept the parameter `phoneNumber`, representing the recipient's phone number.
 - ii. Call the constructor of the superclass `BaseNotification`, passing the parameter `phoneNumber`.
 - (b) Override the method `getType()` to specify the type of notification handled by this class.
 - Return the string "SMS" as the notification type identifier.
 - (c) Override the method `formatMessage(String message)`, whose purpose is to prepare the SMS message content in an appropriate textual format.
 - i. Check the length of the provided parameter `message`.
 - ii. If the message length exceeds 160 characters, shorten the text to the first 157 characters and append "..." at the end.
 - iii. If the message length does not exceed the limit, leave the text unchanged.
 - iv. Store the result in a helper variable named `text`.
 - v. Build the final message content by concatenating the prefix "SMS to ", the value returned by `getRecipient()`, and the shortened message content.
 - vi. Return the created string as the formatted SMS message.
 - (d) Override the method `doSend(String formatted)`, whose purpose is to simulate sending an SMS message.
 - Accept the parameter `formatted`, representing the previously prepared message content.
 - Display the message in the console using `System.out.println`.
 - Prefix the content with [SMS] to indicate the type of notification being sent.
5. Copy the files `NotificationService.java` and `Main.java` into the project and run the program. Check whether the output of the executed program matches the result shown below.

Sending: "Your order #1234 has been shipped!"

[EMAIL] Subject: Notification | To: jan@pj.edu | Body: Your order #1234 has been shipped!

[SMS] SMS to +48 22 58 44 500: Your order #1234 has been shipped!

Sending: ""

[EMAIL] Subject: Notification | To: jan@pj.edu | Body:

[SMS] SMS to +48 22 58 44 500:

Sending: "This is SPAM content"

[EMAIL] Subject: Notification | To: jan@pj.edu | Body: This is SPAM content

[SMS] SMS to +48 22 58 44 500: This is SPAM content

Sending: "Welcome to our store!"

[EMAIL] Subject: Notification | To: jan@pj.edu | Body: Welcome to our store!

[SMS] SMS to +48 22 58 44 500: Welcome to our store!

=== History: OnlineStore ===

Total: 8 | OK: 8 | Failed: 0

[OK] [EMAIL] Your order #1234 has been shipped!

[OK] [SMS] Your order #1234 has been shipped!

[OK] [EMAIL]

[OK] [SMS]

[OK] [EMAIL] This is SPAM content

[OK] [SMS] This is SPAM content

[OK] [EMAIL] Welcome to our store!

[OK] [SMS] Welcome to our store!

6.  Milestone 1 **present the results of your work to the instructor.**
7. In the file `Main.java`, find the comment `TODO 01`. Create an anonymous class implementing the interface `Notification`, and then pass its instance as an argument to the method `addChannel(...)` of the object `service`.
 - (a) Implement the method `send(String message)`, which is responsible for handling notification delivery.
 - Accept the parameter `message`, representing the content of the message.
 - Display a message in the console using `System.out.println`.
 - Prefix the content with `[PUSH]` to indicate the notification channel type.
 - (b) Implement the method `getType()`, whose purpose is to return the identifier of the notification type handled by this implementation.
 - Return the string `"PUSH"` as the name of the notification channel.
8. Run the program and compare the obtained output with the output from the previous point.
9. Declare a public interface `NotificationListener`, which will define the contract for objects reacting to the outcome of the notification-sending process.
 - (a) Add the method `onSuccess(String type, String message)`, whose purpose is to handle the situation in which a notification has been sent successfully.
 - Accept the parameter `type`, representing the notification channel type.
 - Accept the parameter `message`, representing the content of the message that was sent.
 - Leave the implementation of the method to classes implementing the interface.
 - (b) Add the method `onFailure(String type, String message, String reason)`, whose purpose is to handle the situation in which sending a notification has failed.

- Accept the parameter **type**, representing the notification channel type.
 - Accept the parameter **message**, representing the content of the message that was supposed to be sent.
 - Accept the parameter **reason**, representing the reason for the delivery failure.
 - Leave the implementation of the method to classes implementing the interface.
10. In the class `NotificationService.java`, find the comment `TODO 02`. Add class fields responsible for storing and managing the list of objects listening for notification-sending events.
- (a) Declare a private field `listeners` of type `NotificationListener[]`, which will store references to registered event listeners.
 - Initialize the array with an initial size of 10, so that up to ten listeners can be stored without immediately expanding the structure.
 - (b) Add a private field `listenerCount` of type `int`, which will store the current number of registered listeners in the array `listeners`.
 - Initialize the field to 0, to indicate that initially there are no registered listeners.
11. Next, in the same class, find the comment `TODO 03`. Implement the method `addListener(NotificationListener listener)`, whose purpose is to register a new notification event listener and, if necessary, expand the array storing the listeners.
- (a) Check whether the number of stored listeners `listenerCount` is equal to the length of the array `listeners`.
 - (b) If the array is full, expand its capacity manually.
 - i. Create a new array `NotificationListener[]` of size equal to twice the length of the current array `listeners`.
 - ii. Copy all existing elements from the old array to the new one using a `for` loop.
 - iii. Replace the reference stored in the field `listeners` with the new, larger array.
 - (c) Add the provided object `listener` to the array `listeners` at index `listenerCount`.
 - (d) Increase the value of `listenerCount` by one to reflect the number of registered listeners.
12. Then, in the same class, find the comment `TODO 04` and implement the part of the logic responsible for notifying all registered listeners that a notification has been sent successfully.
- (a) Run a `for` loop that iterates through all registered listeners from index 0 to `listenerCount - 1`.
 - (b) For each element of the array `listeners`, call the method `onSuccess(...)`.
 - Pass as the first argument the notification channel type obtained by calling `channels[c].getType()`.
 - Pass as the second argument the content of the sent message `message`.

- As a result of this call, each registered listener is informed that the notification has been sent successfully.
13. In the class `Main.java`, find the comment `TODO 05`. Create an anonymous class implementing the interface `NotificationListener`, and then pass its instance as an argument to the method `addListener(...)` of the object `service`.
- (a) Add private fields `successCount` and `failCount`, which will store the number of successful and unsuccessful notification delivery attempts.
 - (b) Override the method `onSuccess(String type, String message)` to handle the event of a successfully sent notification.
 - Increase the value of the field `successCount` by one.
 - Display a message in the console using `System.out.println`.
 - In the message, include the number of the next success and the notification channel type passed in the parameter `type`.
 - (c) Override the method `onFailure(String type, String message, String reason)` to handle the event of a failed notification.
 - Increase the value of the field `failCount` by one.
 - Display a message in the console using `System.out.println`.
 - In the message, include the number of the next failure, the notification channel type, and the failure reason text passed in the parameter `reason`.
14. Run the program and compare the output to the expected result below:

Sending: "Your order #1234 has been shipped!"

[EMAIL] Subject: Notification | To: jan@pj.edu | Body: Your order #1234 has been shipped!

[AUDIT] OK #1 via EMAIL

[SMS] SMS to +48 22 58 44 500: Your order #1234 has been shipped!

[AUDIT] OK #2 via SMS

[PUSH] Your order #1234 has been shipped!

[AUDIT] OK #3 via PUSH

Sending: ""

[EMAIL] Subject: Notification | To: jan@pj.edu | Body:

[AUDIT] OK #4 via EMAIL

[SMS] SMS to +48 22 58 44 500:

[AUDIT] OK #5 via SMS

[PUSH]

[AUDIT] OK #6 via PUSH

Sending: "This is SPAM content"

[EMAIL] Subject: Notification | To: jan@pj.edu | Body: This is SPAM content

[AUDIT] OK #7 via EMAIL

[SMS] SMS to +48 22 58 44 500: This is SPAM content

[AUDIT] OK #8 via SMS

[PUSH] This is SPAM content

[AUDIT] OK #9 via PUSH

Sending: "Welcome to our store!"

[EMAIL] Subject: Notification | To: jan@pj.edu | Body: Welcome to our store!

[AUDIT] OK #10 via EMAIL

[SMS] SMS to +48 22 58 44 500: Welcome to our store!

[AUDIT] OK #11 via SMS

[PUSH] Welcome to our store!

[AUDIT] OK #12 via PUSH

=== History: OnlineStore ===

Total: 12 | OK: 12 | Failed: 0

[OK] [EMAIL] Your order #1234 has been shipped!

[OK] [SMS] Your order #1234 has been shipped!

[OK] [PUSH] Your order #1234 has been shipped!

[OK] [EMAIL]

[OK] [SMS]

[OK] [PUSH]

[OK] [EMAIL] This is SPAM content

[OK] [SMS] This is SPAM content

[OK] [PUSH] This is SPAM content

[OK] [EMAIL] Welcome to our store!

[OK] [SMS] Welcome to our store!

[OK] [PUSH] Welcome to our store!

15.  Milestone 2 **present the implementation of the introduced changes to the instructor**
16. Declare a functional interface `NotificationFilter`, which will define the contract for mechanisms filtering messages before they are sent.
 - (a) Mark the interface with the annotation `@FunctionalInterface` to indicate that it contains exactly one abstract method and can be used with lambda expressions.
 - (b) Add the method `accept(String message)`, which determines whether a given message should be accepted by the filter.
 - Accept the parameter `message`, representing the content of the message being analyzed.
 - Return a logical value of type `boolean`, indicating whether the message satisfies the filter conditions.
 - Leave the implementation of the method to classes implementing the interface or to lambda expressions.

17. In the class `NotificationService.java`, find the comment `TODO 06`. Add class fields responsible for storing and managing the set of message filters applied before sending notifications.

- (a) Declare a private field `filters` of type `NotificationFilter[]`, which will store references to registered message filters.
 - Initialize the array with an initial size of 10, so that up to ten filters can be stored without immediately expanding the structure.
- (b) Add a private field `filterCount` of type `int`, which will store the current number of filters saved in the array `filters`.
 - Initialize the field to 0, to indicate that initially there are no registered filters.

Next, in the same class, find the comment `TODO 07` and implement the method `addFilter(NotificationFilter filter)`, whose purpose is to register a new message filter and, if necessary, expand the array storing the filters.

- (a) Check whether the number of stored filters `filterCount` is equal to the length of the array `filters`.
 - (b) If the array is full, expand its capacity manually.
 - i. Create a new array `NotificationFilter[]` of size equal to twice the length of the current array `filters`.
 - ii. Copy all existing elements from the old array to the new one using a `for` loop.
 - iii. Replace the reference in the field `filters` with the new, larger array.
 - (c) Add the provided object `filter` to the array `filters` at index `filterCount`.
 - (d) Increase the value of `filterCount` by one to reflect the number of registered filters.
18. In the same class, find the method `sendAll` and the comment `TODO 08`. Below it, implement the logic responsible for verifying the message using all registered filters and handling the case in which the message is rejected.
- (a) Run a `for` loop that iterates through all registered filters from index 0 to `filterCount - 1`.
 - (b) For each filter, call the method `accept(message)` to check whether the message satisfies the conditions of that filter.
 - (c) Check whether the result of the method `accept(...)` is equal to `false`.
 - i. If the message has been rejected by the filter, create a variable `reason` storing the textual description of the rejection reason.
 - ii. Display a console message informing that the message has been blocked.
 - iii. Run a `for` loop that iterates through all available notification channels from index 0 to `channelCount - 1`.
 - A. Add an entry to the event history by creating an object `Result` containing information about the failure, the channel type, the content of the message, and the reason for rejection.

- B. Run another `for` loop that iterates through all registered listeners from index 0 to `listenerCount - 1`.
 - C. For each listener, call the method `onFailure(...)`.
 - Pass as the first argument the notification channel type obtained by `channels[c].getType()`.
 - Pass as the second argument the message content `message`.
 - Pass as the third argument the rejection reason stored in the variable `reason`.
 - iv. End the current operation by executing the `return` statement, in order to stop further processing of the message.
19. In the class `Main.java`, find the comment `TODO 09` and add below it three message filters to the object `service`, using lambda expressions implementing the functional interface `NotificationFilter`.
- (a) Add a filter that checks whether the message is not an empty or blank string.
 - Define a lambda expression accepting the parameter `msg`.
 - In the body of the expression, call the method `isBlank()` on the provided text.
 - Return the logical negation of this result, so that only messages containing non-whitespace characters are accepted.
 - (b) Add a filter limiting the maximum message length.
 - Define a lambda expression accepting the parameter `msg`.
 - Check whether the text length returned by `msg.length()` is less than or equal to 200.
 - Return the result of this comparison as the filter's decision.
 - (c) Add a filter blocking messages containing a specific word.
 - Define a lambda expression accepting the parameter `msg`.
 - Convert the message content to lowercase by calling `msg.toLowerCase()`.
 - Check whether the transformed text does not contain the string `"spam"` using the method `contains(...)`.
 - Return the logical negation of this check result in order to reject messages containing the specified word.

20. Sending: "Your order #1234 has been shipped!"

[EMAIL] Subject: Notification | To: jan@pj.edu | Body: Your order #1234 has been shipped!

[AUDIT] OK #1 via EMAIL

[SMS] SMS to +48 22 58 44 500: Your order #1234 has been shipped!

[AUDIT] OK #2 via SMS

[PUSH] Your order #1234 has been shipped!

[AUDIT] OK #3 via PUSH

Sending: ""

BLOCKED: Message rejected by filter

[AUDIT] FAIL #1 via EMAIL -- Message rejected by filter

[AUDIT] FAIL #2 via SMS -- Message rejected by filter

[AUDIT] FAIL #3 via PUSH -- Message rejected by filter

Sending: "This is SPAM content"

BLOCKED: Message rejected by filter

[AUDIT] FAIL #4 via EMAIL -- Message rejected by filter

[AUDIT] FAIL #5 via SMS -- Message rejected by filter

[AUDIT] FAIL #6 via PUSH -- Message rejected by filter

Sending: "Welcome to our store!"

[EMAIL] Subject: Notification | To: jan@pj.edu | Body: Welcome to our store!

[AUDIT] OK #4 via EMAIL

[SMS] SMS to +48 22 58 44 500: Welcome to our store!

[AUDIT] OK #5 via SMS

[PUSH] Welcome to our store!

[AUDIT] OK #6 via PUSH

=== History: OnlineStore ===

Total: 12 | OK: 6 | Failed: 6

[OK] [EMAIL] Your order #1234 has been shipped!

[OK] [SMS] Your order #1234 has been shipped!

[OK] [PUSH] Your order #1234 has been shipped!

[FAIL] [EMAIL] (reason: Message rejected by filter)

[FAIL] [SMS] (reason: Message rejected by filter)

[FAIL] [PUSH] (reason: Message rejected by filter)

[FAIL] [EMAIL] This is SPAM content (reason: Message rejected by filter)

[FAIL] [SMS] This is SPAM content (reason: Message rejected by filter)

[FAIL] [PUSH] This is SPAM content (reason: Message rejected by filter)

[OK] [EMAIL] Welcome to our store!

[OK] [SMS] Welcome to our store!

[OK] [PUSH] Welcome to our store!

21. 🚩 Milestone 3 **present the result of your work to the instructor**

22. In the class `NotificationService.java`, find the comment `TODO 10`, and then implement the method `getByChannel(String channel)`, whose purpose is to return the results of notification deliveries belonging to the specified communication channel.

- (a) Accept the parameter `channel`, which specifies the type of notification channel used to filter the results.
- (b) Call the method `getFiltered(...)` of the object `history`, passing to it a filtering

function in the form of a lambda expression.

- In the lambda expression, accept the parameter `result`, representing a single history entry.
 - For each element, call the method `getChannel()` to retrieve the notification channel type stored in that result.
 - Compare the obtained value with the parameter `channel` using the method `equals(...)`.
 - Return the result of the comparison as the filter decision indicating whether a given element should be included in the result.
- (c) Return the array `Result[]` returned by the method `history.getFiltered(...)` as the result of this method.
23. In the class `Main.java`, find the comment `TODO 11` and implement the code fragment responsible for displaying the results of notifications sent only through the `EMAIL` channel.
- (a) Display in the console a section header indicating that results for the `EMAIL` channel are being presented.
- Use the method `System.out.println`.
 - Add a newline character at the beginning of the string to separate this section from the earlier program output.
- (b) Retrieve the results of notifications sent through the `EMAIL` channel.
- Call the method `getByChannel("EMAIL")` on the object `service`.
 - Store the returned array `Result[]` in the variable `emailResults`.
- (c) Iterate through all elements of the array `emailResults` using a `for` loop.
- Iterate from index `0` to `emailResults.length - 1`.
- (d) For each element of the array, display its textual representation.
- Call `System.out.println(emailResults[i])`.
 - As a result, the `toString()` method of the object `Result` will be used.

24. Run the program and check whether the output is:

```
--- EMAIL only ---
[OK] [EMAIL] Your order #1234 has been shipped!
[FAIL] [EMAIL] (reason: Message rejected by filter)
[FAIL] [EMAIL] This is SPAM content (reason: Message rejected by filter)
[OK] [EMAIL] Welcome to our store!
```

25. In the class `NotificationService.java`, find the comment `TODO 12`. Implement the static method `sort(Result[] results, ResultComparator comparator)`, whose purpose is to sort the array of `Result` objects using the provided comparator and return a new sorted copy of the array.

- (a) Create a new array `Result[]` named `sorted` with a length equal to the length of the array `results`, in order to prepare a copy of the input data.
 - (b) Copy all elements from the array `results` into the array `sorted`.
 - i. Run a `for` loop that iterates from index 0 to `results.length - 1`.
 - ii. In each iteration, assign the element from the array `results` to the corresponding position in the array `sorted`.
 - (c) Implement the **Bubble Sort** algorithm, which will use the provided comparator to compare elements.
 - i. Run an outer `for` loop that iterates from index 0 to `sorted.length - 2`.
 - ii. In each iteration, run an inner `for` loop that iterates from index 0 to `sorted.length - 2 - i`.
 - iii. For each pair of adjacent elements, call the method `comparator.compare(sorted[j], sorted[j + 1])`.
 - iv. Check whether the result of the comparison is greater than 0, which means that the elements are in the wrong order.
 - v. If the condition is satisfied, swap the elements.
 - Store the element `sorted[j]` in a temporary variable `temp`.
 - Assign the element `sorted[j + 1]` to position `sorted[j]`.
 - Assign the value of the variable `temp` to position `sorted[j + 1]`.
 - (d) After sorting is complete, return the array `sorted` as the result of the method.
26. In the class `Main.java`, find the comment `TODO 13`. Implement the code fragment responsible for sorting and displaying notification results by timestamp in descending order, from the most recent to the oldest.
- (a) Display a section header in the console indicating that the results are being sorted by time.
 - Use the method `System.out.println`.
 - Place a newline character at the beginning to separate the section from the earlier program output.
 - (b) Call the static method `sort(...)` of the class `NotificationService` to obtain a sorted array of results.
 - i. Pass as the first argument the array of results obtained by calling the method `service.getSuccessful()`.
 - ii. Pass as the second argument a comparator defined in the form of a lambda expression.
 - In the lambda expression, accept two parameters representing the compared `Result` objects.
 - Retrieve the timestamps of both results by calling `getTimestamp()`.
 - Call the method `compareTo(...)` in reverse order to obtain descending sorting, that is, from the most recent to the oldest.

- (c) Store the returned array in the variable `sorted`.
- (d) Iterate through all elements of the array `sorted` using a `for` loop.
 - Iterate from index 0 to `sorted.length - 1`.
- (e) For each element of the array, display its textual representation.
 - Call `System.out.println(sorted[i])`.
 - As a result, the `toString()` method of the object `Result` will be used.

27. The final output should look as follows:

```
Sending: "Your order #1234 has been shipped!"
[EMAIL] Subject: Notification | To: jan@pj.edu | Body: Your order #1234 has been shipped!
[AUDIT] OK #1 via EMAIL
[SMS] SMS to +48 22 58 44 500: Your order #1234 has been shipped!
[AUDIT] OK #2 via SMS
[PUSH] Your order #1234 has been shipped!
[AUDIT] OK #3 via PUSH
```

```
Sending: ""
BLOCKED: Message rejected by filter
[AUDIT] FAIL #1 via EMAIL -- Message rejected by filter
[AUDIT] FAIL #2 via SMS -- Message rejected by filter
[AUDIT] FAIL #3 via PUSH -- Message rejected by filter
```

```
Sending: "This is SPAM content"
BLOCKED: Message rejected by filter
[AUDIT] FAIL #4 via EMAIL -- Message rejected by filter
[AUDIT] FAIL #5 via SMS -- Message rejected by filter
[AUDIT] FAIL #6 via PUSH -- Message rejected by filter
```

```
Sending: "Welcome to our store!"
[EMAIL] Subject: Notification | To: jan@pj.edu | Body: Welcome to our store!
[AUDIT] OK #4 via EMAIL
[SMS] SMS to +48 22 58 44 500: Welcome to our store!
[AUDIT] OK #5 via SMS
[PUSH] Welcome to our store!
[AUDIT] OK #6 via PUSH
```

```
=== History: OnlineStore ===
Total: 12 | OK: 6 | Failed: 6
[OK] [EMAIL] Your order #1234 has been shipped!
[OK] [SMS] Your order #1234 has been shipped!
[OK] [PUSH] Your order #1234 has been shipped!
```

[FAIL] [EMAIL] (reason: Message rejected by filter)
[FAIL] [SMS] (reason: Message rejected by filter)
[FAIL] [PUSH] (reason: Message rejected by filter)
[FAIL] [EMAIL] This is SPAM content (reason: Message rejected by filter)
[FAIL] [SMS] This is SPAM content (reason: Message rejected by filter)
[FAIL] [PUSH] This is SPAM content (reason: Message rejected by filter)
[OK] [EMAIL] Welcome to our store!
[OK] [SMS] Welcome to our store!
[OK] [PUSH] Welcome to our store!

--- EMAIL only ---

[OK] [EMAIL] Your order #1234 has been shipped!
[FAIL] [EMAIL] (reason: Message rejected by filter)
[FAIL] [EMAIL] This is SPAM content (reason: Message rejected by filter)
[OK] [EMAIL] Welcome to our store!

--- Sorted by timestamp (newest first) ---

[OK] [EMAIL] Welcome to our store!
[OK] [SMS] Welcome to our store!
[OK] [PUSH] Welcome to our store!
[OK] [SMS] Your order #1234 has been shipped!
[OK] [PUSH] Your order #1234 has been shipped!
[OK] [EMAIL] Your order #1234 has been shipped!

28. ★ Milestone 4 present the result of your work to the instructor